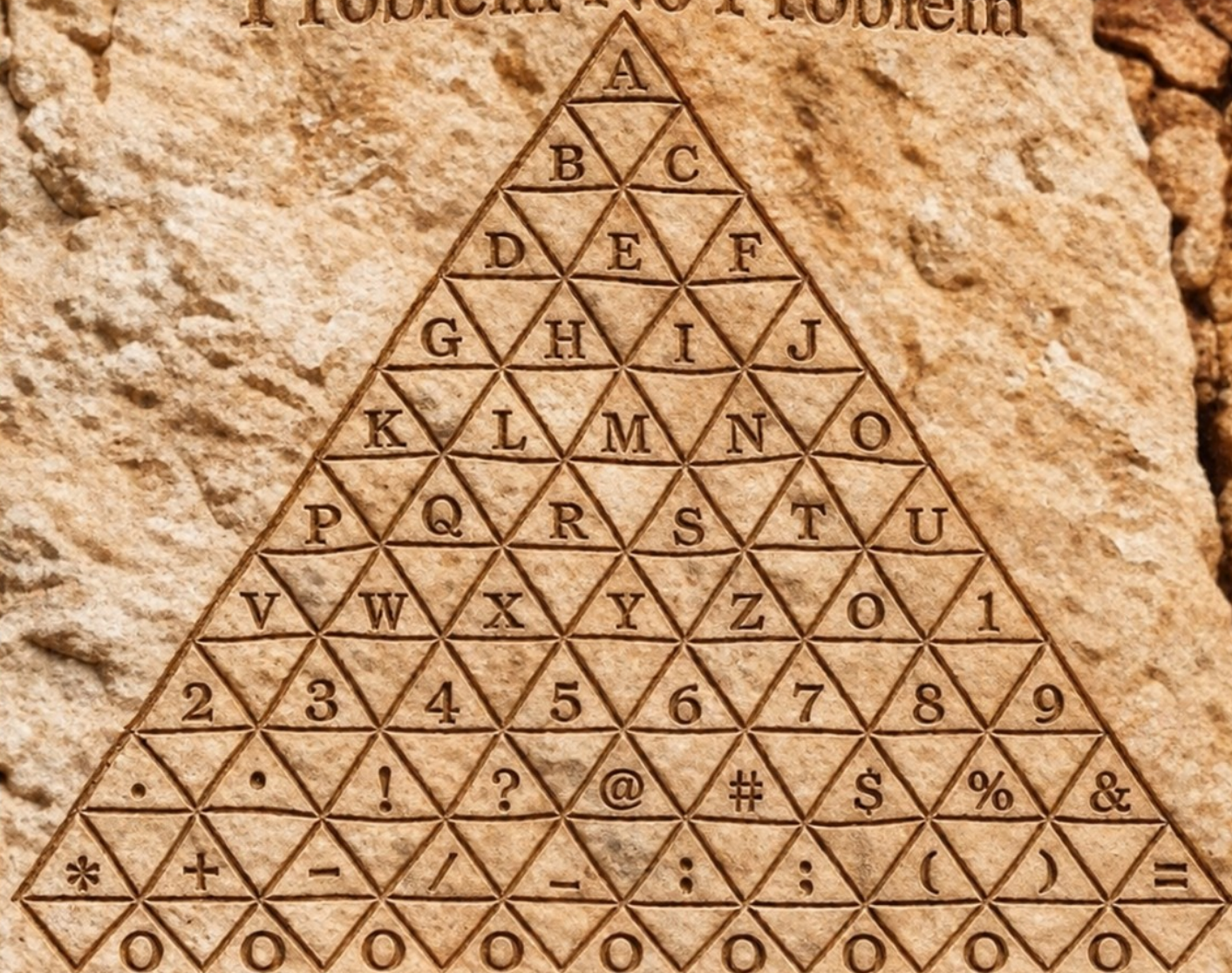


$P = NP$
Problem No Problem



ERITRO OVV TZGGVMYB
VIRGIL LEE GATTENBY

$P = NP$

PROBLEM NO PROBLEM

Current-State Existential Folding and Verified Semantic Closure

Virgil Lee Gattenby

18 September 2026

Abstract

I start with the Boolean state in hand. Choose one variable and ask what remains when it is 0 and when it is 1. Join those two restrictions with existential OR, then use that result as the next current state. Repeat until no assignment coordinates remain. I want to know whether every step preserves the truth needed to answer SAT, and what work the machine must do to carry that truth forward.

The construction follows ten stages, M0–M9: Type-Set Zero, Distinction, Current number, VPA rotation, Hinge siblings, Mirror closure, Existential fold, Current successor, Ordinal succession, and Local closure. Mirror closure records shared truth and unmatched residue; the fold eliminates the selected coordinate. Each successor, rather than the original formula, supplies the next step.

Lean verifies the Boolean fold identity, its one-step existential meaning, and repeated semantic elimination for every finite variable count. A separate benchmark checks 17,902 normalized three-variable CNFs through 53,706 folds. An independently implemented Boolean-mask checker agrees at the initial, intermediate, and terminal states. Replacing OR with AND triggers the negative control.

This establishes finite semantic Local Closure, not a conventional proof that $P = NP$. The present truth-table implementation starts with 2^n entries. A polynomial-time result still needs a compact exact representation and polynomial bounds for every operation across the full succession. Counting the folds does not pay for what is inside them.

Claim boundary: finite semantic correctness CLOSED; finite replay VERIFIED; standard-machine polynomial time and conventional $P = NP$ NOT ESTABLISHED.

Contents

1. The question and the exact claim
 2. Boolean algebra, CNF encoding, and the current-state fold
 3. Current-state construction
 4. Lemmas and the local closure theorem
 5. Machine-checked semantic core
 6. Finite implementation, independent checker, and receipts
 7. Complexity: where the proof stops
 8. Source authority and reproducibility
 9. Methodological note
 10. Conclusion
- References

“We must know, we shall know.”

— David Hilbert [13], translated from German

1. The question and the exact claim

Boolean satisfiability asks whether a Boolean formula has at least one satisfying assignment. Cook, Levin, and Karp placed SAT and related problems at the center of the modern P-versus-NP landscape [1–3]. In the conventional formulation, P contains decision problems solvable in deterministic polynomial time, while NP contains decision problems whose proposed solutions can be verified in polynomial time [4–6]. Because SAT is NP-complete, a deterministic polynomial-time SAT algorithm would imply $P = NP$.

I keep correctness and cost separate. An algorithm can return the right answer every time and still carry an exponential amount of work. The first obligation is to show that the fold preserves existential truth. The second is to account for the size of the state and the work needed to update it.

SEMANTIC QUESTION

If one Boolean coordinate is eliminated at a time from the current state, does the successor retain exactly the original state's existential truth over the eliminated coordinates?

COMPLEXITY QUESTION

Can that current state be represented and updated on a standard deterministic machine with total cost bounded by a polynomial in the length of the original input?

The first question is answered here. The second remains open. I keep them separate because a correct answer does not tell me what it cost to obtain it. Both questions deserve an answer; proving one does not dispose of the other.

The original state appears once, at bootstrap. Every later step operates on the successor produced by the preceding step.

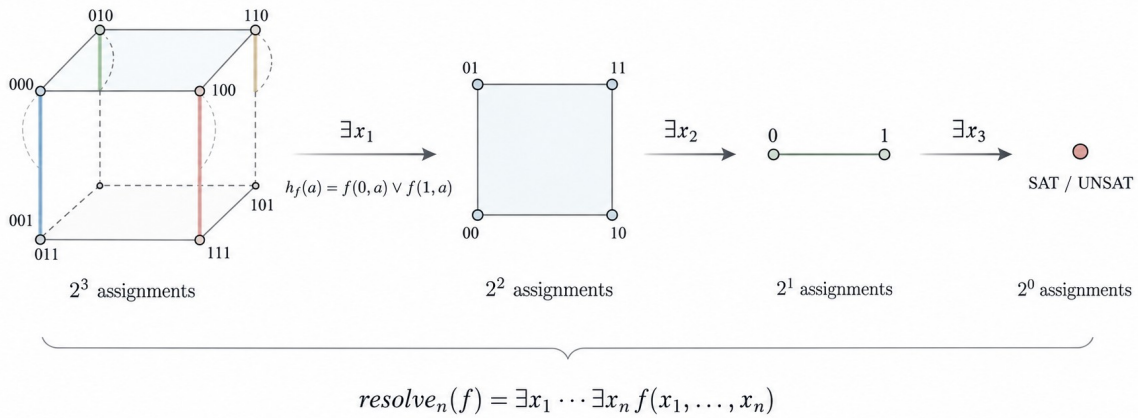


Figure 1. Exact existential folding of Boolean assignment space. The three-variable assignment cube is reduced by successive existential folds from 2^3 states to the terminal SAT/UNSAT value.

2. Boolean algebra, CNF encoding, and the current-state fold

I first encode the CNF as a Boolean-valued expression, then define the restrictions, the fold, and the shared-truth accounting that connects them.

2.1 Boolean quotient algebra

For a finite CNF F on variables $x_1, \dots, x_n$, the governing source uses the Boolean quotient algebra

$$\mathfrak{B}_n = \mathbb{Z}[x_1, \dots, x_n] / \langle x_i^2 - x_i : 1 \leq i \leq n \rangle.$$

Literal, clause, and formula values are encoded by

$$\begin{aligned} L(x_i) &= x_i, & L(\neg x_i) &= 1 - x_i, \\ C_j &= 1 - \prod_{l \in C_j} (1 - L(l)), & f_F &= \prod_j C_j. \end{aligned}$$

On Boolean assignments these expressions evaluate to Boolean truth. The existential interpretation below applies to Boolean-valued current states. It is not licensed for arbitrary integer-valued elements merely because the notation looks cooperative.

Shannon's switching algebra provides historical context for Boolean relations [7]. Symbolic representations such as binary decision diagrams show why a simple Boolean operation need not have a small representation [8]. I do not claim novelty for the OR identity or existential elimination. The contribution documented here is the current-state construction and its checked semantic result.

2.2 One selected coordinate

Fix the coordinate order $x_1, \dots, x_n$. At step i , where $0 \leq i < n$, the current state f_i has already eliminated $x_1, \dots, x_i$; the next coordinate is x_{i+1} . Define its two restrictions

$$a = f_i|_{x_{i+1}=0}, \quad b = f_i|_{x_{i+1}=1}.$$

The existential fold is

$$\mathcal{E}_{x_{i+1}}(f_i) = a \vee b = a + b - ab.$$

For $a, b \in \{0, 1\}$, the four cases are 0, 1, 1, 1. Therefore $a + b - ab$ is exactly Boolean OR. Pointwise, for every assignment r to the coordinates that remain,

$$\mathcal{E}_{x_{i+1}}(f_i)(r) = 1 \Leftrightarrow \exists c \in \{0, 1\} \text{ such that } f_i(c, r) = 1.$$

One coordinate disappears. Its existential meaning does not.

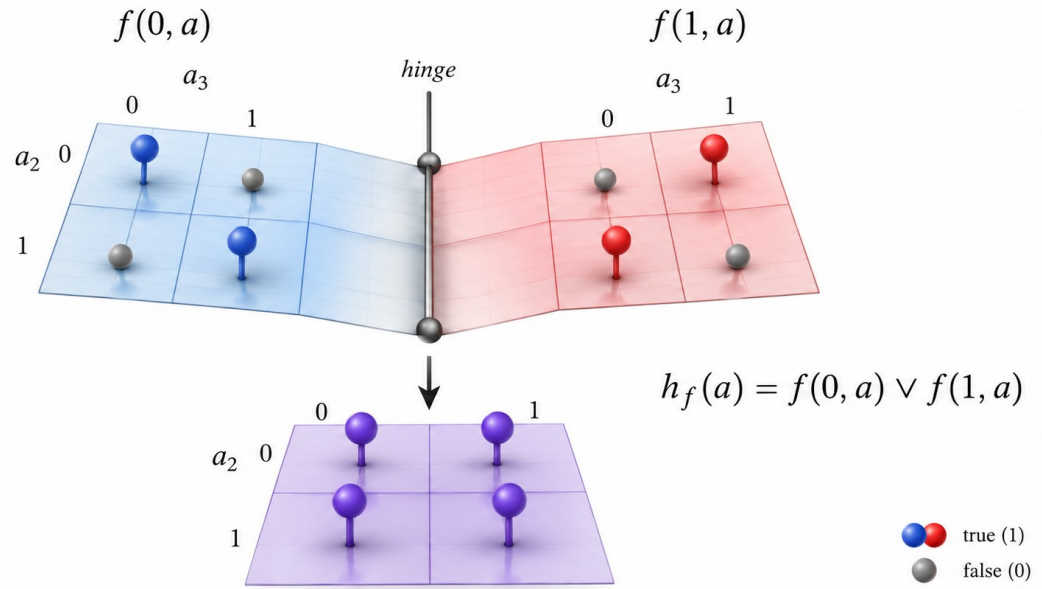


Figure 2. One-step Boolean hinge. The sibling truth sheets $f(0, a)$ and $f(1, a)$ are paired on the selected coordinate and resolved by $h_f(a) = f(0, a) \vee f(1, a)$.

2.3 Mirror receipt before action

M5 records the shared and unmatched parts before M6 performs existential elimination. For Boolean a and b ,

$$\text{shared} = ab, \quad \text{left residue} = a(1-b), \quad \text{right residue} = (1-a)b,$$

$$\text{shared} + \text{left residue} + \text{right residue} = a + b - ab = a \vee b.$$

M5 is the provenance receipt. M6 is the existential action. The receipt and the transaction are related; they are not the same line item.

2.4 A complete fold, with the remainder visible

Consider $F = (x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$. The two clauses ask different things of x_1 . Setting x_1 to zero leaves x_2 ; setting it to one leaves x_3 . Both restrictions belong to the same current formula. That shared origin matters before any cancellation is discussed.

$$f_0 = (x_1 + x_2 - x_1x_2)(1 - x_1 + x_1x_3).$$

$$a = f_0|_{x_1=0} = x_2, \quad b = f_0|_{x_1=1} = x_3.$$

The shared truth is x_2x_3 . The left residue is $x_2(1-x_3)$, and the right residue is $(1-x_2)x_3$. These three Boolean pieces are disjoint: where both restrictions are true, the shared piece carries that truth once; where only one is true, its residue survives. Nothing needs to disappear merely to make the expression shorter.

$$\begin{aligned} f_1 &= x_2x_3 + x_2(1-x_3) + (1-x_2)x_3 \\ &= x_2 + x_3 - x_2x_3. \end{aligned}$$

Now eliminate x_2 from f_1 , rather than returning to f_0 . The restrictions are x_3 and 1. Their shared part is x_3 ; the left residue is zero and the right residue is $1-x_3$. The account still records both possibilities, but existential truth has become constant.

$$f_2 = x_3 + 1 - x_3 = 1, \quad f_3 = 1 + 1 - 1 = 1.$$

The fixed three-coordinate schedule therefore ends in SAT. The last fold removes an assignment coordinate even though no variable remains in the function's support. This is the distinction behind Lemma 3: support sizes are 3, 2, 0, 0, while remaining coordinate counts are 3, 2, 1, 0. A fold can finish its administrative work after the answer is already constant.

For the opposite terminal, take $G = x_1 \wedge \neg x_1$. Its encoding $x_1(1-x_1)$ is zero in the quotient algebra. Both restrictions are zero, their fold is zero, and the result is UNSAT. For the negative control, take $H = x_1$: its restrictions are 0 and 1. OR returns 1; replacing OR with AND returns 0 and loses a satisfying assignment. The checker has something definite to object to.

These calculations illustrate the algebraic restrictions and folds. They do not supply an implementation of the separate mirror-certificate protocol.

3. Current-state construction

The ten stages below organize the current-state construction [16]. They specify the order of operations. Section 4 gives the four supporting lemmas and the local closure theorem; Section 5 identifies the corresponding Lean statements. A stage label is not an additional theorem.

Source authority. The locked MATHIC identified in Section 8 governs the construction order, stage names, and declared scope. This manuscript supplies a conventional exposition. Its notation conventions are explicit below; they do not amend the locked artifact or extend its Lean coverage.

Index convention. The source writes the selected coordinate as x_i in the transition from f_i to f_{i+1} . Here f_0 is the initial state and x_1 is the first coordinate, so the selected coordinate at state i is written x_{i+1} . Only the coordinate label is shifted. The source's strict rank decrease applies when that coordinate is present, as its accompanying explanation specifies. The paper displays that condition explicitly and also records non-increase for an absent coordinate.

Operational roles. Value V_i carries f_i . Perception P_i compares the two restrictions of that same current state and records their shared and unmatched truth. Action A_i commits the existential fold; the resulting function becomes V_{i+1} . The rotation $\rho^3=\text{id}$ concerns the three role labels. It does not assert that three computations restore the original formula.

A successor imprint σ_{i+1} denotes the committed successor record; f_{i+1} is its Boolean mathematical payload. The ordinal label τ_i records the dependency order. Fold₀ names the commitment of a completed

fold, not an additional Boolean operator. The $T_9 \rightarrow T_0$ notation records completion and return to the next admission stage; it does not turn a UNSAT value of zero into one.

The source requires a provenance certificate before antipodal cancellation. Its local chart does not supply a complete executable certificate format or verification predicate. The shared/residue identity below therefore explains the Boolean accounting only. A complete implementation of the certificate protocol remains outside the formalized semantic core.

The construction follows the ten-stage sequence defined in [16]:

$$M0 \rightarrow M1 \rightarrow M2 \rightarrow M3 \rightarrow M4 \rightarrow M5 \rightarrow M6 \rightarrow M7 \rightarrow M8 \rightarrow M9 \rightarrow T0.$$

The strict role order is Value $\rightarrow$ Perception $\rightarrow$ Action $\rightarrow$ emergent Value. The successor is emergent from the completed action; it is not copied from an earlier state.

M0. Type-Set Zero

Type: base + law · **Atomic:** reference · **Reflection:** — · **Fold:** — · **Checksum:** τ -axis

$$\mathfrak{B}_n = \mathbb{Z}[x_1, \dots, x_n] / \langle x_i^2 - x_i \rangle$$

Admits the governing base, Boolean type, lawful operators, ordinal reference, and provenance before distinction.

M1. Distinction

Type: δ · **Atomic:** $0 \leftrightarrow 1$ · **Reflection:** armed · **Fold:** — · **Checksum:** typed

$$\delta : 0 \neq 1, \quad \{0,1\} \subset \mathfrak{B}_n$$

Creates the first lawful Boolean difference. The distinction is typed before VPA rotation begins.

M2. Current number

Type: $\mathfrak{B}_n$ · **Atomic:** $N_i = \langle V, P, A \rangle$ · **Reflection:** — · **Fold:** — · **Checksum:** origin

$$N_i = \langle V_i, P_i, A_i \rangle, \quad V_i = f_i$$

Constitutes the exact formula-state being operated on now as one current number.

M3. VPA rotation

Type: $\rho^3 = \text{id}$ · **Atomic:** $V \rightarrow P \rightarrow A$ · **Reflection:** orientation · **Fold:** wait · **Checksum:** VPA ✓

$$\rho(V)=P, \quad \rho(P)=A, \quad \rho(A)=V, \quad \rho^3=\text{id}$$

Performs the atomic Value-to-Perception-to-Action cycle.

M4. Hinge siblings

Type: same T_0 · Atomic: $V_i^0 \bowtie V_i^1$ · Reflection: reflect · Fold: wait · Checksum: current ✓

$$V_i^0 = f_i|_{x_{i+1}=0}, \quad V_i^1 = f_i|_{x_{i+1}=1}$$

Reflects the exact current state into the two restrictions associated with the selected Boolean coordinate.

M5. Mirror closure

Type: provenance · Atomic: shared once · Reflection: cancel $\perp$ · Fold: carry Δ · Checksum: mirror ✓

$$S = ab, \quad \Delta_0 = a(1-b), \quad \Delta_1 = (1-a)b$$

Certifies mirror provenance, retains shared lawful truth once, and preserves every unmatched residue. Only a certified antipodal component may cancel. Desired-answer matching is not a certificate: failed pairing returns HOLD or $\perp$ (bottom), with no fabricated successor. The decomposition above records Boolean truth; by itself it does not establish a provenance certificate. M5 records and checks; M6 performs existential elimination.

M6. Existential fold

Type: $x^2 = x$ · Atomic: $a+b-ab$ · Reflection: siblings joined · Fold: Fold₀ · Checksum: truth ✓

$$\mathcal{E}_{x_{i+1}}(f_i) = f_i|_{x_{i+1}=0} + f_i|_{x_{i+1}=1} - f_i|_{x_{i+1}=0} f_i|_{x_{i+1}=1}$$

Applies exact Boolean existential OR and immediately folds the two restrictions into one successor imprint.

M7. Current successor

Type: normalize · Atomic: σ_{i+1} · Reflection: recorded · Fold: $\mu_{i+1} \leq \mu_i$ · Checksum: rank ✓

$$f_{i+1} = \text{Normalize}_{(x^2=x)}(\mathcal{E}_{x_{i+1}}(f_i)), \quad \mu_i = |\text{Var}(f_i)|$$

Normalizes the fold and binds it as the next exact current state. Here $\text{Var}(f_i)$ is the set of coordinates on which f_i depends. Support never grows; if x_{i+1} belongs to $\text{Var}(f_i)$, then $\mu_{i+1} \leq \mu_i - 1$. The remaining assignment coordinates number $n-i$, even when some are absent from the support.

M8. Ordinal succession

Type: $\tau_i \rightarrow \tau_{i+1}$ · Atomic: next current · Reflection: repeat lawful · Fold: $\leq n$ folds · Checksum: continuity ✓

$$f_k \equiv \exists x_1 \dots \exists x_k f_0, \quad f_{i+1} \text{ is the only input to the next fold}$$

Repeats the same atomic measure on the exact successor along the ordinal axis; it never restarts from the immutable original formula.

M9. Local closure

Type: $T_9 \rightarrow T_0$ · Atomic: $f_n \in \{0,1\}$ · Reflection: provenance kept · Fold: terminal · Checksum: PASS

$$f_n = 1 \Leftrightarrow \exists x_1 \dots \exists x_n f_0 = 1 \Leftrightarrow F \text{ is satisfiable}$$

Returns the exact Boolean terminal after at most n coordinate eliminations. The typed T_9 -to- T_0 return completes the local continuity checksum.

4. Lemmas and the local closure theorem

Let $F_0 = F$ and let $f_0 = f_F$ be its encoding in the Boolean quotient algebra. In the fixed coordinate order $x_1, \dots, x_n$, define for $0 \leq i < n$

$$f_{i+1} = \text{Normalize}_{(x^2=x)}(\mathcal{E}_{x_{i+1}}(f_i)).$$

Lemma 1. Exact existential fold

For every assignment to the remaining coordinates,

$$f_{i+1} = 1 \Leftrightarrow f_i|_{x_{i+1}=0} = 1 \vee f_i|_{x_{i+1}=1} = 1 \Leftrightarrow \exists x_{i+1} f_i = 1.$$

Proof. Substitute the two Boolean restrictions into $a+b-ab$ and inspect the four Boolean cases. The result is OR, so it is true exactly when at least one restriction is true. ■

Lemma 2. Continuity invariant

$$f_k \equiv \exists x_1 \dots \exists x_k f_0.$$

Proof. The $k=0$ case is immediate. Assume the statement for k . Lemma 1 gives $f_{k+1} \equiv \exists x_{k+1} f_k$. Substituting the induction hypothesis yields the stated $k+1$ form. Each successor comes only from the exact current state. ■

Lemma 3. Atomic rank descent

$$\begin{aligned} \mu_i &= |\text{Var}(f_i)|, & \mu_{i+1} &\leq \mu_i; \\ x_{i+1} \in \text{Var}(f_i) &\Rightarrow \mu_{i+1} &\leq \mu_i - 1. \end{aligned}$$

Proof. Restriction removes x_{i+1} and introduces no new dependency. The sum and product of the two restrictions depend only on remaining coordinates; normalization using $x^2 = x$ preserves their function. Thus support cannot grow, and it strictly decreases when the selected coordinate was present. The assignment carrier loses one coordinate at every fold, whether or not the function depended on it. After n folds no assignment coordinate remains, so the result is scalar. Support size bounds productive eliminations; n bounds all coordinate folds. Neither bounds representation size or machine work. ■

Lemma 4. Terminal exactness

$$f_n \in \{0,1\},$$

$$f_n = 1 \Leftrightarrow \exists x_1 \dots \exists x_n f _ F = 1 \Leftrightarrow F \text{ is satisfiable},$$

$$f_n = 0 \Leftrightarrow F \text{ is unsatisfiable}.$$

Proof. After every coordinate has been eliminated, no assignment argument remains. The continuity invariant reduces the terminal to the existential truth of the original Boolean formula. ■

Theorem 1. Finite semantic local closure

For every natural number n , including $n = 0$, and every Boolean-valued function f on $A_n = \text{Assignment}(n)$, the recursively defined resolver satisfies

$$\text{resolve}_n(f) = \text{true} \Leftrightarrow \exists a \in A_n, f(a) = \text{true}.$$

Proof. Induct on n . At $n = 0$, $\text{Assignment}(0)$ is Unit, with sole assignment $()$. The resolver returns $f()$, so its result is true exactly when a satisfying assignment exists. For $n + 1$, every assignment is a pair (b, a) , where b is Boolean and a belongs to $\text{Assignment}(n)$. The hinge is true at a exactly when $f(\text{false}, a)$ or $f(\text{true}, a)$ is true. Apply the induction hypothesis to this hinge function: the recursive result is true exactly when some a and some b satisfy $f(b, a)$. This proves the equivalence for $n + 1$. ■

The construction is semantically exact on every declared finite Boolean assignment carrier. This closes the semantic question stated in Section 1. It does not, by itself, close the standard-machine complexity question.

4.1 Formula progression

The argument follows the sequence below, from encoding the input to interpreting the terminal value.

Encoding	$L(x_j) = x_j, \quad L(\neg x_j) = 1 - x_j,$ $C_j = 1 - \prod_l \in C_j (1 - L(l)), \quad f_0 = \prod_j C_j.$
Restrictions · M4	$a_i = f_i _{x_{i+1}=0}, \quad b_i = f_i _{x_{i+1}=1}.$
Accounting · M5	$S_i = a_i b_i,$ $\Delta_i^0 = a_i(1 - b_i), \quad \Delta_i^1 = (1 - a_i)b_i.$
Successor · M6–M8	$f_{i+1} = \text{Normalize}(S_i + \Delta_i^0 + \Delta_i^1)$ $= \text{Normalize}(a_i + b_i - a_i b_i) \equiv \exists x_{i+1} f_i,$
Invariant	$f_k \equiv \exists x_1 \dots \exists x_k f_0.$
Terminal · M9	$f_n = 1 \Leftrightarrow F \text{ is satisfiable};$ $f_n = 0 \Leftrightarrow F \text{ is unsatisfiable}.$

Lemmas 1–4 establish the algebraic argument. Theorem 1 states the recursive Boolean-function result represented in Lean. The finite replay separately checks 17,902 formulas through 53,706 folds. Polynomial machine cost remains the additional hypothesis of Proposition 2 in Section 7.

5. Machine-checked semantic core

The Lean file `verification/tryx-lean/TryxProof.lean` deliberately formalizes the semantic core without assuming a constant-cost representation. It defines

$$\begin{aligned} \text{Assignment}(0) &= \text{Unit}, & \text{Assignment}(n+1) &= \text{Bool} \times \text{Assignment}(n), \\ \text{hinge}(f, a) &= f(\text{false}, a) \text{ OR } f(\text{true}, a), \\ \text{resolve}(0, f) &= f(), & \text{resolve}(n+1, f) &= \text{resolve}(n, \text{hinge}(f)). \end{aligned}$$

Lean proves three named PNP statements:

Lean theorem	Checked statement	Role in the paper
TRYX.PNP.boolean_fold_algebra	Integer encoding of $a \text{ OR } b$ equals $a+b-ab$ for Boolean a, b .	M6 algebra
TRYX.PNP.hinge_exact	$\text{hinge}(f, a) = \text{true}$ iff some Boolean branch makes f true.	M4–M6 one-step semantics
TRYX.PNP.resolve_correct	$\text{resolve}(n, f) = \text{true}$ iff some assignment in Assignment n makes f true.	M7–M9 repeated closure

The accepted proof commit is 496c55eb628588d1afbb49233ab8b91ddb82a271. Workflow run 34445813111 records acceptance by the Lean default kernel and the independent nanoda checker. The theorem axiom reports contain `propext`, as recorded by the repository evidence file.

The archived verification description also records comparison against reference statements and permitted axioms, with reference placeholders isolated from the solution. The repository pins the Lean toolchain and package versions. These checks concern the named formal statements; they do not certify every construction label or computational-cost claim in the manuscript.

The Lean translation does not formalize the polynomial quotient representation as an efficient data structure, the CNF encoder, certified mirror-provenance packets, or a compact successor machine. A proof should not receive credit for a theorem it never stated. Mathematics does not grade on intention.

5.1 What the formal statements cover

The three Lean statements have specific mathematical jobs. `boolean_fold_algebra` covers the OR identity used at M6. `hinge_exact` connects the restrictions at M4 to existential truth at M6. `resolve_correct` covers repeated semantic elimination and its terminal interpretation at M7–M9. Their correspondence is to these operations, not to every label in the construction.

M0’s integer quotient and CNF encoding are developed in Section 2; they are not the data structures checked by these three Lean statements. M1’s Boolean distinction is represented by the Boolean carrier. M2 and M3 organize roles around the current state. M5 adds provenance obligations beyond the

shared/residue identity. The successor and terminal record fields likewise require their own implementation checks.

The connection from the quotient calculation to the Lean carrier is evaluation. Evaluate each polynomial on a Boolean assignment and identify 0 and 1 with false and true. The relation $x^2=x$ holds on those assignments. Restriction commutes with evaluation, and $a+b-ab$ evaluates to Boolean OR. Inductively, an exact normalized polynomial fold and the Boolean hinge therefore produce the same truth function. This paragraph supplies the mathematical bridge; the cited Lean declarations do not themselves formalize that polynomial evaluator.

The distinction permits a precise claim: the recorded Lean evidence supports the named semantic statements. It does not certify an unspecified certificate format, an efficient quotient implementation, or the whole MATHIC merely by association.

6. Finite implementation, independent checker, and receipts

“Beware of bugs in the above code; I have only proved it correct, not tried it.”

— Donald E. Knuth [14]

6.1 Enumerated population

The replay uses three Boolean variables. For each variable, a clause may contain the positive literal, the negative literal, or no literal. Excluding the all-absent choice gives

$$3^3 - 1 = 26$$

distinct nonempty, non-tautological clauses with no repeated variable. Formulas are sets of zero through four distinct clauses:

$$\sum_{k=0}^4 C(26,k) = 17,902 \text{ formulas.}$$

The zero-clause case is the empty conjunction, whose value is true. It is included in the population. The fold count records coordinate-elimination levels, not individual machine operations.

Each formula carries three coordinates, so the replay performs

$$17,902 \times 3 = 53,706 \text{ coordinate folds.}$$

The Python 3 replay represents each initial state as an eight-entry Boolean truth table, then folds $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. Every successor comes from the immediately preceding table. The original terminal comparison shares the initial evaluation with the fold engine, so agreement alone would not independently check that evaluation.

The independent checker generates clauses separately through ternary inclusion/sign choices, evaluates literals with its own code, builds assignment masks, and intersects the clause masks. At every elimination level it compares the current table with direct quantification of that separately calculated mask. This checks the initial evaluation as well as the intermediate and terminal states.

6.2 Independent negative control

I do not ask the fold engine to mark its own homework. The independent checker constructs its oracle separately and compares the initial, intermediate, and terminal states. As a negative control, it replaces OR with AND on the formula x_1 and requires a disagreement. It gets one.

“If the checker never says no, you may have earned a golf clap”

— Virgil Lee Gattenby

6.3 Recorded results

Receipt field	Recorded value
Admissible clauses	26
Formulas	17,902
Coordinate folds	53,706
SAT terminals	16,241
UNSAT terminals	1,661
Failures	0
Independent disagreements	0
OR $\rightarrow$ AND negative control	DETECTED

The fixture tells us whether this implementation agrees with the independent oracle on the declared population. It does not tell us that the representation scales polynomially. The counts, intermediate states, and negative control are there so the reader can check the same claim.

“trust is welcome, receipts are forever”

— Virgil Lee Gattenby

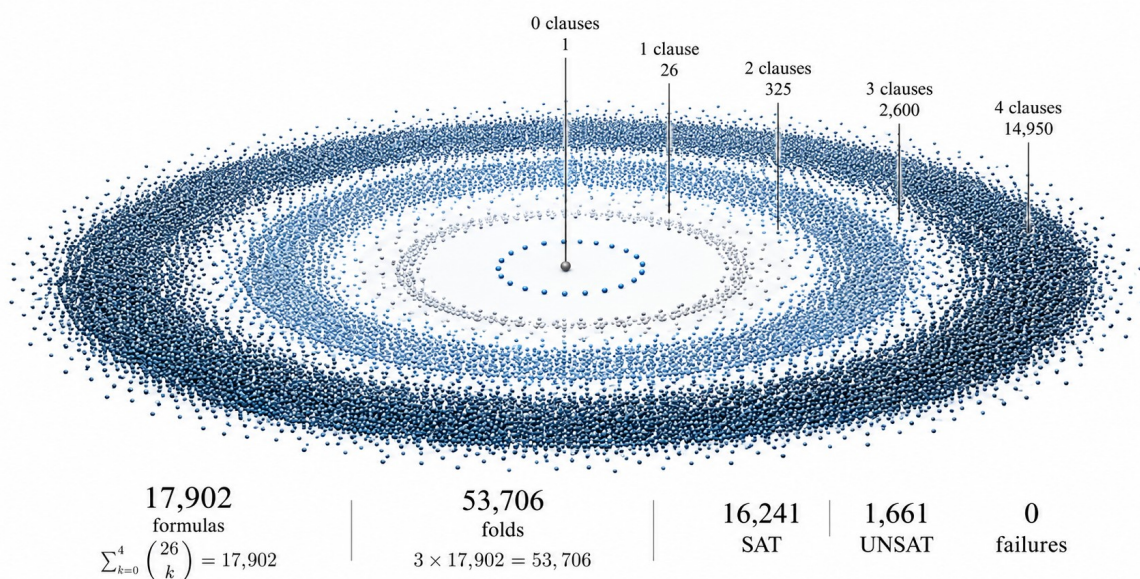


Figure 3. Finite benchmark field. The declared three-variable fixture contains 17,902 formulas and 53,706 folds, with 16,241 SAT cases, 1,661 UNSAT cases, and zero recorded failures in the primary replay.

7. Complexity: where the proof stops

The semantic theorem says that repeated existential folding returns the correct Boolean terminal. The conventional P-versus-NP problem asks whether a deterministic standard machine can obtain that terminal in polynomial time measured against the length of the original input [4–6].

For the explicit truth-table implementation, the starting state contains 2^n entries. After one fold it contains 2^{n-1} entries, then 2^{n-2} , and so on. The number of paired combinations after the table already exists is

$$2^{n-1} + 2^{n-2} + \dots + 1 = 2^n - 1.$$

There are only n elimination levels, but the objects carried through them may be exponentially large.

A conventional promotion requires a concrete exact representation and polynomial bounds, in original input length, for every relevant operation:

1. initial input encoding;
2. the size of every intermediate successor state;
3. restriction or substitution;
4. multiplication or equivalent symbolic composition, when used;
5. normalization or canonicalization;
6. equality and compatibility tests;
7. storage and traversal in the selected machine model;
8. terminal decision; and
9. witness reconstruction, if a witness is claimed.

Promotion obligation	Status
Polynomial representation-size bound	OPEN
Polynomial restriction/substitution cost	OPEN
Polynomial symbolic composition cost	OPEN
Polynomial normalization/canonicalization cost	OPEN
Polynomial witness reconstruction	OPEN
Total standard-machine polynomial runtime	OPEN
Conventional P = NP	NOT ESTABLISHED

Proposition 2. Conditional polynomial-time implementation

Let L be the encoded length of a CNF input, with its distinct variables indexed so that $n \leq L$. Suppose a uniform deterministic algorithm encodes its initial state exactly in polynomial time; produces at most n exact existential successors; uses a representation whose size stays polynomial in L ; performs each complete successor update in polynomial time in L ; and reads the terminal Boolean value in polynomial time. Each update bound includes every normalization, comparison, certificate check, storage operation, and other required subroutine. Then that algorithm decides SAT in polynomial time.

Proof. Exactness and Theorem 1 give the correct SAT/UNSAT terminal. If encoding, each complete update, and terminal readout cost at most polynomials $p_0(L)$, $p_1(L)$, and $p_2(L)$, the total cost is bounded by

$$\begin{aligned} T(L) &\leq p_0(L) + n \cdot p_1(L) + p_2(L) \\ &\leq p_0(L) + L \cdot p_1(L) + p_2(L). \end{aligned}$$

This is polynomial in L . Since SAT is NP-complete [1–6], the hypotheses would imply $P = NP$. ■

The implication is established here; its implementation hypotheses are not. In particular, the explicit truth table does not satisfy the polynomial-size hypothesis as n grows. The task is to exhibit a representation and update procedure with those bounds, then prove the bounds for that procedure.

The fold is exact in the stated Boolean setting. Its machine cost is a separate question.

“Closed is a math status. Pay me is a different apartment”

— Virgil Lee Gattenby

This paper does not claim to overcome relativization, natural-proofs barriers, or other general limits on proof methods [11,12]. Those questions become controlling only if the argument reaches the conventional complexity claim. The present result establishes finite semantic closure.

The broader complexity context is surveyed in [9,10]. The semantic theorem here does not supply a polynomial bound for the representation or its operations.

Reference roles. Cook, Levin, and Karp [1–3] support the reduction framework that makes a polynomial SAT algorithm consequential. The standard treatments [4–6] fix the computational model and complexity claim. Shannon [7] supplies historical context for Boolean algebra, while Bryant [8] supplies a concrete symbolic-representation comparison. Fortnow [9,10] frames the wider problem; the barrier papers [11,12] constrain particular proof strategies. None of these references verifies the present MATHIC or substitutes for its implementation bounds.

8. Source authority and reproducibility

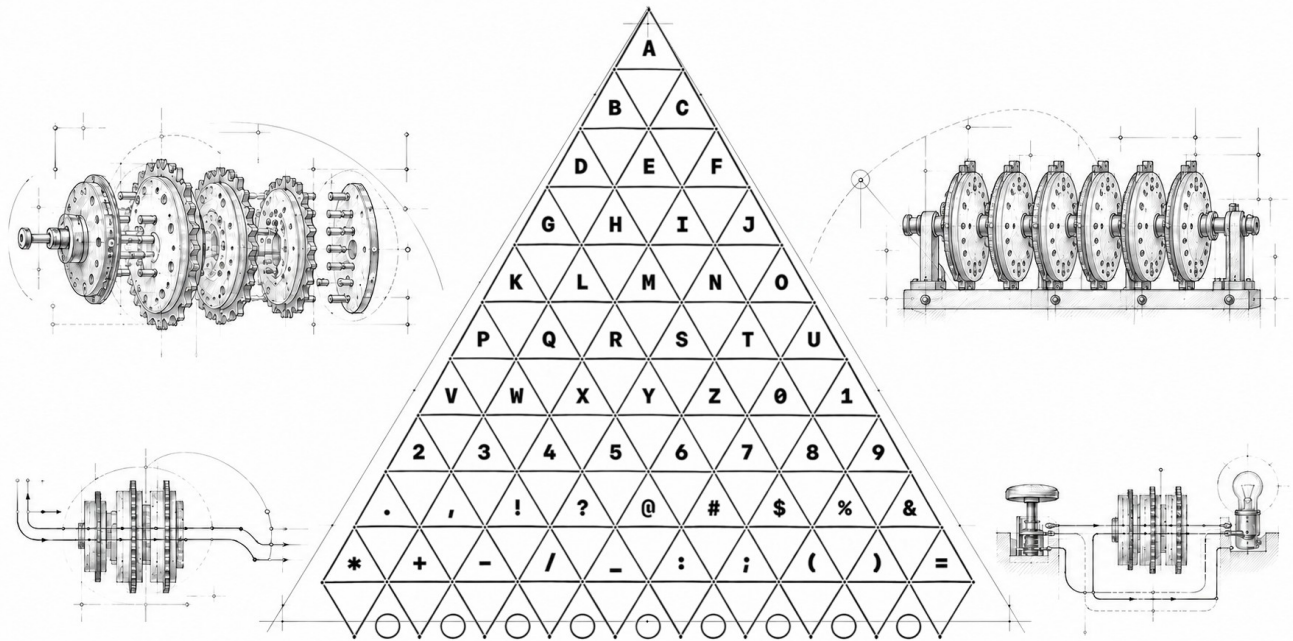


Figure 4. Code-breaking demonstration instrument. The triangular character field is framed by Enigma rotor, Bombe drum, signal-path, and lamp-circuit studies. The instrument is demonstrative, not a complexity proof.

The construction is specified in [16]. The source identifiers and computational artifacts in [15] support reproduction of its algebra, formal statements, and recorded checks.

The Triadic E6B (TRI-E6B) is the companion visual cipher instrument built around the locked PNP MATHIC. Its triangular design assigns source states to upright cells and transformed states to inverted cells, making their relationship available for inspection. The instrument provides an operational setting for the construction; its cipher behavior does not establish the polynomial-time bounds required here.

Artifact	Authority / receipt
Locked PNP MATHIC	charts/pnp/ TRYX.PNP.LOCAL.CLOSURE.MATHIC.SCORE.LY RIC.SUCCESSOR.260905.LOCKED.html
Locked MATHIC SHA-256	95b63f3ce280a494ac1be28d222bd03bc1da010e17e c6be3a1ba3438687073fb
Governing source body	publications/pnp-mathic/sources/ TRYX.PNP.CONTINUITY.CLOSURE.REINTRODU CTION.BENCHMARK.260831.235122Z.md
Governing source SHA-256	9a611f7c1ae0d3a1f299411d83146dfbd7ddf8b7bd6 93b9fddd20f7d1badaa3a
Lean semantic core	verification/tryx-lean/TryxProof.lean
Accepted proof commit	496c55eb628588d1afbb49233ab8b91ddb82a271
Accepted workflow	GitHub Actions run 34445813111
Replay and independent check	publications/pnp-mathic/replay.py and independent_check.py

From the repository root, run `python3 publications/pnp-mathic/check_package.py` without Python's `-O` flag. The checker verifies source hashes and reproduces the stored replay and independent-check receipts.

9. Methodological note

I used AI for source searches, computational replay, candidate Lean statements, and adversarial checks. I remain responsible for the mathematical claims. For verification instructions and linked build evidence, see the companion [Problem No Problem README](#). AI drift is the worst part of the process, but it is manageable. I manage it by holding the source fixed, comparing revisions against it, and rerunning the checks before accepting a change. A fluent explanation does not replace a proof or a reproducible result.

10. Conclusion

I have given an exact current-state existential fold for finite Boolean functions. The M0–M9 construction admits a typed state, distinguishes its Boolean restrictions, records shared and unmatched truth, and carries each exact fold into the next state. The four-lemma continuity argument reaches the same finite semantic terminal proved in Lean.

The executable evidence agrees across 17,902 formulas and 53,706 folds, including every intermediate state checked by the independent oracle. The negative control is detected. The receipts are specific, reproducible, and fenced from claims they do not support.

What remains is specific: exhibit a compact exact successor representation and prove polynomial bounds for its size and every required operation across the full succession. I cannot substitute a short list of steps for that bound. Until it is supplied, finite semantic Local Closure is closed and conventional $P = NP$ remains open.

References

- [1] S. A. Cook, “The Complexity of Theorem-Proving Procedures,” *Proceedings of STOC ’71*, pp. 151–158, 1971. DOI: 10.1145/800157.805047.
- [2] L. A. Levin, “Universal Sequential Search Problems,” *Problems of Information Transmission* 9(3), pp. 265–266, 1973.
- [3] R. M. Karp, “Reducibility Among Combinatorial Problems,” in *Complexity of Computer Computations*, pp. 85–103, 1972. DOI: 10.1007/978-1-4684-2001-2_9.
- [4] S. Cook, “The P versus NP Problem,” official Millennium Prize problem description, Clay Mathematics Institute, 2000.
- [5] S. Arora and B. Barak, *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [6] M. R. Garey and D. S. Johnson, *Computers and Intractability*. W. H. Freeman, 1979.
- [7] C. E. Shannon, “A Symbolic Analysis of Relay and Switching Circuits,” *Transactions of the AIEE* 57(12), pp. 713–723, 1938. DOI: 10.1109/T-AIEE.1938.5057767.
- [8] R. E. Bryant, “Graph-Based Algorithms for Boolean Function Manipulation,” *IEEE Transactions on Computers* C-35(8), pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819.
- [9] L. Fortnow, “The Status of the P versus NP Problem,” *Communications of the ACM* 52(9), pp. 78–86, 2009. DOI: 10.1145/1562164.1562186.
- [10] L. Fortnow, “Fifty Years of P vs. NP and the Possibility of the Impossible,” *Communications of the ACM* 65(1), pp. 76–85, 2022. DOI: 10.1145/3460351.
- [11] T. Baker, J. Gill, and R. Solovay, “Relativizations of the $P = ? NP$ Question,” *SIAM Journal on Computing* 4(4), pp. 431–442, 1975. DOI: 10.1137/0204037.
- [12] A. A. Razborov and S. Rudich, “Natural Proofs,” *Journal of Computer and System Sciences* 55(1), pp. 24–35, 1997. DOI: 10.1006/jcss.1997.1494.
- [13] D. Hilbert, “Naturerkennen und Logik,” address delivered in Königsberg, 8 September 1930; published in *Die Naturwissenschaften* 18, pp. 959–963, 1930. English translation: MacTutor History of Mathematics, University of St Andrews, <https://mathshistory.st-andrews.ac.uk/Biographies/Hilbert/>.
- [14] D. E. Knuth, “Notes on the van Emde Boas Construction of Priority Deques: An Instructive Use of Recursion,” memorandum to Peter van Emde Boas, 29 March 1977; quotation provenance confirmed in Knuth’s Stanford FAQ. <https://cs.stanford.edu/~knuth/faq.html>.
- [15] V. L. Gattenby, “Problem No Problem Mathic: current-state existential folding and verified semantic closure,” TRYX-MATHIC research manuscript and reproducibility package, 2026. Repository: <https://github.com/TRYX-Relay/TRYX-MATHIC>.
- [16] V. L. Gattenby, TRYX.PNP.LOCAL.CLOSURE.MATHIC.SCORE.LYRIC.SUCCESSOR.260905.LOCKED.html, locked PNP MATHIC authority, 5 September 2026.